Neural Networks from First Principles: The Perceptron Rule and a Hand-Derived Backpropagation MLP

Nithin D H
Indian Institute of Technology Kanpur (IIT-Kanpur)
Kanpur, India

Abstract

We implement two neural learners in plain NumPy, writing every forward and backward computation ourselves rather than using a deep-learning library.

The first is Rosenblatt's perceptron with a step activation, trained on a 20-point, two-class dataset from three random initialisations. Two runs find a perfect separating line, after 4 and 17 passes. The third still misclassifies one point after the 20-pass budget, which shows that initialisation changes how fast *the rule converges, not whether a separator exists.*

The second is a 4–5–3 multilayer perceptron (tanh hidden layer, softmax output, mean-squared-error loss) trained with per-sample SGD on Iris. We derive the MSE gradient through the softmax Jacobian and confirm it against finite differences (relative error 2×10^{-9}*). After 1,000 epochs the network reaches* ***98.3% training and 100% test accuracy*** *(30/30) with only 43 parameters.*

1. Introduction

Why build networks from scratch? Libraries hide the two ideas that make neural networks work: a local weight-update rule, and the chain rule that propagates error backwards through layers. Implementing both by hand shows what each line of a training loop does, and gives a reference against which library code can be checked.

What we build. (i) The perceptron, the simplest trainable neuron, whose convergence on linearly separable data is guaranteed by the perceptron convergence theorem [3, 4]. (ii) A one-hidden-layer network trained by backpropagation [5], on the classic Iris data [1].

2. Related Work

Rosenblatt's perceptron [4] updates weights only on misclassified samples; Novikoff [3] bounded the number of mistakes by $(R/\gamma)^2$, where R is the data radius and γ the margin. Rumelhart *et al.* [5] popularised backpropagation for multilayer networks, and LeCun *et al.* [2] collected the practical advice (input standardisation, small random initial weights) followed here.

3. Method

3.1. Perceptron

Each input is augmented with a bias term, $\tilde{x} = [1, x_1, x_2]$. The prediction is $\hat{y} = \mathbb{1}[w^\top \tilde{x} \geq 0]$, and every sample triggers the update

$$w \leftarrow w + \eta\,(y - \hat{y})\,\tilde{x}, \qquad \eta = 0.01. \tag{1}$$

Weights start from $\mathcal{N}(0, 1)$ draws. Training runs for 20 passes over the data, from seeds 42, 7 and 123.

3.2. Multilayer perceptron

Architecture. 4 inputs $\to$ 5 tanh units $\to$ 3 softmax outputs (43 parameters). Weights are initialised from $0.1 \cdot \mathcal{N}(0, 1)$ and biases at zero.

Loss and gradients. With one-hot targets Y and outputs $A_2 = \mathrm{softmax}(Z_2)$, the loss is $L = \frac{1}{NC}\sum(A_2 - Y)^2$. Let $G = \partial L/\partial A_2 = \frac{2}{NC}(A_2 - Y)$. Because softmax couples its outputs, the gradient with respect to the logits needs its Jacobian:

$$\frac{\partial L}{\partial Z_2} = A_2 \odot \Big(G - \textstyle\sum_c G_c A_{2,c}\Big). \tag{2}$$

This error is then propagated through W_2 and through $\tanh'(z) = 1 - \tanh^2(z)$ to the first layer. The pairing is easy to get wrong: $A_2 - Y$ is the gradient for *cross-entropy* with softmax, not for MSE. An earlier version of this implementation used it while reporting MSE; the corrected version reaches the same final accuracy. A finite-difference check on W_2 gives a relative error of 2.2×10^{-9}.

Training. Iris (150 samples, 3 classes) is split 80/20 with stratification, and features are standardised with training statistics. Training uses per-sample SGD with learning rate 0.01 for 1,000 epochs, reshuffling every epoch.

4. Results and Discussion

4.1. Perceptron: initialisation sets the pace

Two seeds reach zero training errors (Table 1, Fig. 1), which proves the data are linearly separable. The convergence theorem therefore guarantees that seed 123 would also converge

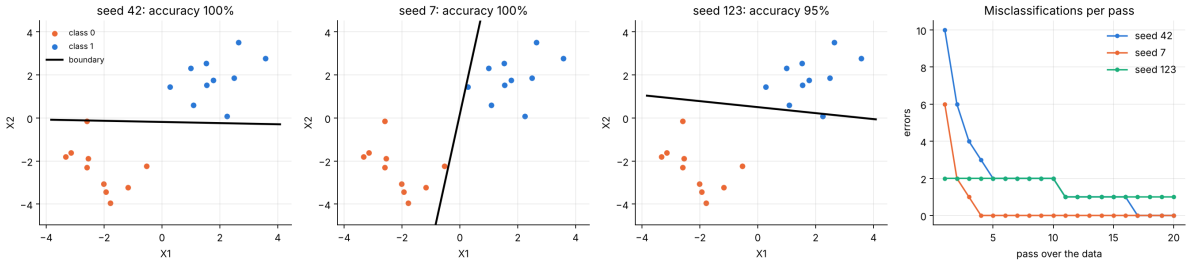


Figure 1. Perceptron decision boundaries after 20 passes for three initialisations (left three panels) and misclassifications per pass (right).

Table 1. Perceptron runs (20 samples, 20 passes, $\eta = 0.01$).

Seed	Final $w = [w_0, w_1, w_2]$	Accuracy	First error-free pass
42	$[0.195,\ 0.028,\ 1.060]$	100%	17
7	$[0.011,\ 0.384,\ -0.063]$	100%	4
123	$[-0.689,\ 0.190,\ 1.364]$	95%	not within 20

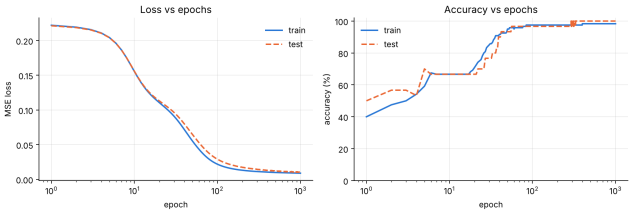


Figure 2. MLP loss (left) and accuracy (right) per epoch; the epoch axis is logarithmic.

Table 2. MLP training checkpoints (MSE loss; accuracy in %).

Epoch	Train loss	Test loss	Train acc.	Test acc.
1	0.222	0.221	40.0	50.0
10	0.155	0.156	66.7	66.7
50	0.055	0.065	95.0	93.3
100	0.022	0.029	97.5	96.7
500	0.010	0.012	98.3	100.0
1000	0.009	0.011	**98.3**	**100.0**

given more passes; it simply started further from a separating direction and ran out of budget one point short.

The three separators are also very different. Seed 42 learns an almost horizontal line ($|w_2| \gg |w_1|$), while seed 7 learns a steep one ($|w_1| \gg |w_2|$). The perceptron stops at *any* separator, not the widest-margin one, so the solution depends on where it starts. A max-margin method such as an SVM would remove that dependence.

4.2. MLP on Iris

Training is smooth and has three phases (Fig. 2). From about epoch 6 to 19, accuracy sits at 66.7%, which is what two of the three classes correct would give. Accuracy then rises quickly (90% at epoch 36, 95% at epoch 50) and is slowly refined after that. Table 2 gives checkpoints.

At the end, 2 of 120 training samples are misclassified, consistent with the known *versicolor/virginica* overlap. All 30 test samples are correct, and the test confusion matrix is diagonal (10/10/10). Train and test losses stay close throughout, so this 43-parameter network shows no sign of overfitting.

Caveat. With 30 test samples, one error moves test accuracy by 3.3 points, so "100%" is consistent with a true accuracy in the high 90s. Repeated splits or cross-validation would give a tighter estimate.

5. Conclusion & Future Work

A hand-written perceptron and a hand-derived backpropagation network both behave as theory predicts: the perceptron converges on separable data at a speed that depends on its initialisation, and a 43-parameter MLP classifies Iris almost perfectly once the MSE gradient is derived correctly through the softmax.

Next steps. (i) Compare MSE and cross-entropy losses on convergence speed; (ii) add mini-batches, momentum and a learning-rate schedule; (iii) average accuracy over repeated stratified splits; and (iv) visualise the five hidden units' activations to see how the network separates the classes.

Acknowledgement. This work was carried out as part of EE708 Fundamentals of Data Science and Machine Intelligence at IIT Kanpur under the supervision of Dr. Rajesh M. Hegde.

References

[1] R. A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.

[2] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller. Efficient BackProp. In *Neural Networks: Tricks of the Trade*, Springer, 1998.

[3] A. B. J. Novikoff. On convergence proofs on perceptrons. In *Symposium on the Mathematical Theory of Automata*, 1962.

[4] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[5] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.